Mini-Proyecto 2

Por:

David Gutierrez

Universidad Javeriana

Ingeniería de Sistemas

Fund. de Algoritmos y Programación

Cali

2024

En el curso de Fundamentos de Algoritmos y Programación, se llevó a cabo un mini proyecto orientado al desarrollo de un juego de pelea simple programado en el idioma Python para correr en una consola de computadora. Esto con el objetivo de aplicar los conceptos teóricos adquiridos a lo largo de los primeros 2 cortes del semestre además que también fomentan la creatividad y el trabajo en equipo entre los participantes. A través de la programación, aplicamos las diversas técnicas de diseño de algoritmos, estructuras de control y manejo de variables, lo que resultó en la creación de un juego interactivo que simula un combate entre personajes ficticios por turnos. Además, también se aplicó una forma inicial de diseño para el usuario, requiriendo un dibujo simple para una mejor inmersión y entendimiento para el jugador.

El juego consiste en peleas por turnos individuales entre 2 jugadores, primera instancia en seleccionar a uno de los diversos personajes para usar, cada uno teniendo su oportunidad para escoger. Siendo estos personajes distintivos no sólo en nombre sino también en habilidades y diseño visual. Luego, inicia una selección por azar de estadísticas para cada jugador, estás siendo los puntos de vida y el daño base. Con el mismo sistema de azar, se elige cual de los 2 jugadores tomará la iniciativa e iniciará atacando. Cada personaje cuenta con 3 habilidades con diferentes características pero ordenadas de modo que las primeras sean las más precisas y las últimas sean las más dañinas. Un sistema de azar determinará si la habilidad golpea, reduciendo la vida del enemigo, o falla. El primer jugador en reducir la vida del otro a 0 o inferior, gana.

Con el objetivo de tener unas bases para llevar a cabo el objetivo, se inició el proyecto definiendo el portal creativo del aplicativo. Con esto en mente, se inició dibujando en la consola los personajes y definiendo las reglas generales del juego como las habilidades de cada personaje. Además, haciendo pruebas con el código para definir qué implementaciones funcionarán en caso tal de ser necesitadas. Con esto definido, agregar aspectos parcializandolos con las funciones permitió llevar un orden para terminar el programa general y en caso de haber un problema con dicha función, tener sus componentes organizados.

Reglas del juego

Nomenclaturas:

Ataque: el daño es un valor aleatorio entre 10 y 200**0.**
Defensa: un valor aleatorio entre la mitad y 9/10 del ataque del otro jugador.
Daño: el daño total es el resultado de restar el ataque del jugador que esté atacando *(la forma de operar el daño varía según la habilidad seleccionada)* y la defensa del jugador atacado, esto se hace así, para evitar daños nulos(mientras que el ataque no tenga probabilidad de fallar).
Salud: la salud, para tener un juego más largo y que no se acabe al primer turno se estableció en 2000 puntos.
probabilidad de fallar: 2 de las 4 habilidades tienen una probabilidad de fallar (lo que mostrará el mensaje que notifica que ha hecho 0 de daño), esta probabilidad varía según el ataque.
Rango: el rango es la diferencia entre el valor mínimo y valor máximo
ignorar defensa:la capacidad de ignorar en la operación del daño el valor de la defensa, es decir, reemplazar por 0 el valor de ella, hay 2 habilidades que tienen esta característica.

Explicación de habilidades:

1. Puñetazo: el puñetazo es el ataque con menor rango, su daño depende del ataque del jugador más un número aleatorio entre 1 y 10, menos la defensa del otro jugador.
2. Patada giratoria: la patada es el ataque con mayor rango, ya que este esta entre 0.5 y 1.5 del ataque del jugador, menos la defensa del otro jugador, en el caso que el 0.5 del ataque sea menor a la defensa, su ataque fallará.
3.Tiro al aire: el tiro es un ataque que tiene un 50% de probabilidad de fallar, en caso de que acierte, tiene un 50% de probabilidad de ignorar la defensa del otro jugador, por lo que en realidad tiene un 25% de ignorar la defensa enemiga.
4.Puñalada: la puñalada es una habilidad que no toma en cuenta el ataque del jugador, sino que este toma un valor entre 50, y 100, sin embargo esta habilidad ignora la defensa del otro jugador.

Algoritmo

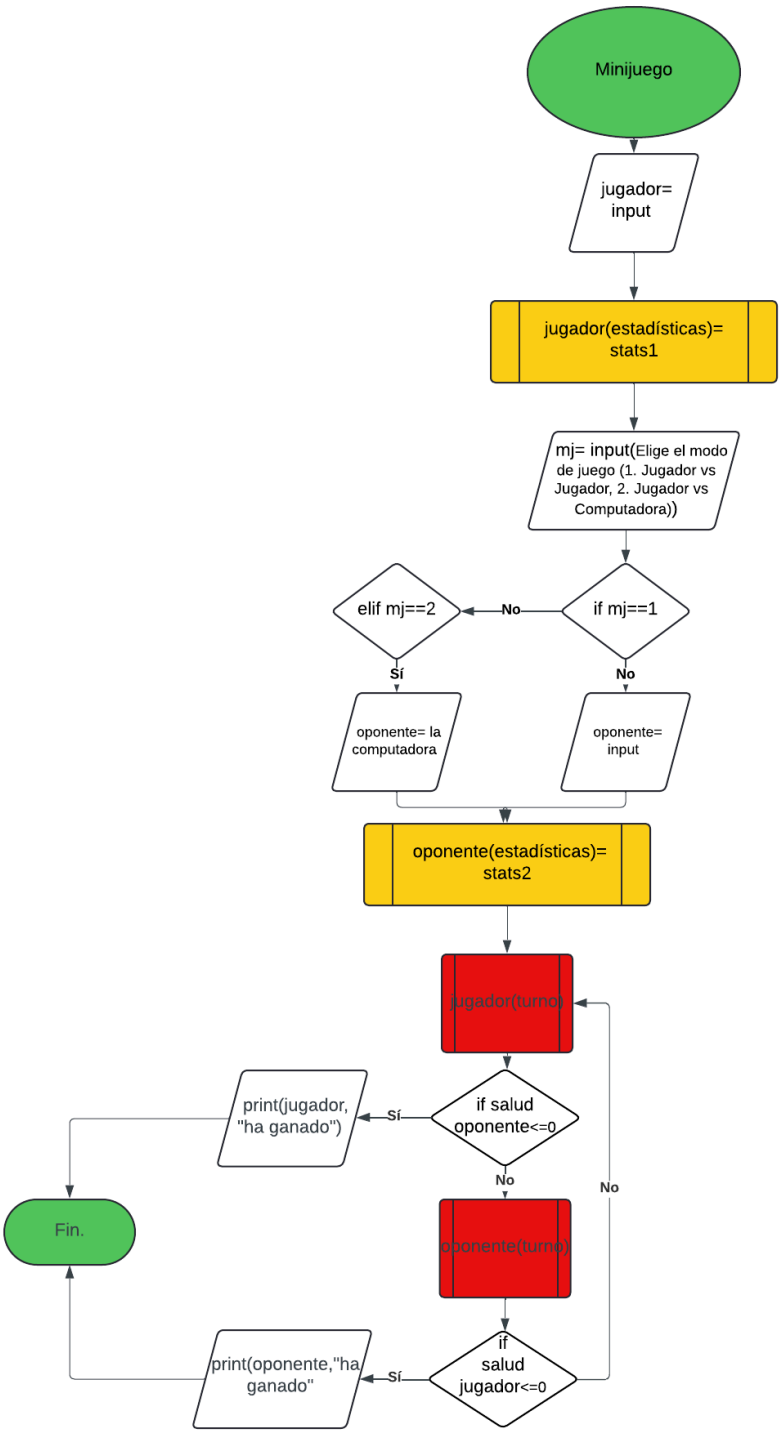
```python
from IPython.display import clear_output
import random
clear_output()
Accion=[] #El primer digito es Multiplicador de daño, el segundo es precision

def Bienvenida():
  global J1, J2
  J1=input("Bienvenido!, Cual es tu nombre jugador 1?: ")
  J2=input("Cual es tu nombre jugador 2?: ")
  input("Presiona Enter para continuar")
  clear_output()
  return

Bienvenida()
def Dado():
  D=random.randint(1,10)
  return D
def Bruno():
  print("     _Ω_")
  print("     @     ")
  print("    /|\/7 ==   ")
  print("    / \    ")
def BrunoHab():
  global Accion
  print("""Selecciona tu accion
    1. Disparo
    2. Rafaga
    3. Balacera""")
  while True:
    Actuar=int(input())
    if Actuar==1:
      Accion=[1,8]#El primer digito es Multiplicador de daño, el segundo es precision
      print("Disparo")
      break
    elif Actuar==2:
      Accion=[3,6]
      print("Rafaga")
      break
    elif Actuar==3:
      print("Balacera")
      Accion=[6,2]
      break
    else:
      print("Opcion no valida")
def Pepe():
  print("     _-_")
  print("     @     ")
  print("    /|\~ı~=   ")
  print("    / \    ")
def PepeHab():
  global Accion
  print("""Selecciona tu accion
    1. Corte
    2. Puñalada
    3. Boliar Machete""")
  while True:
    Actuar=int(input())
    if Actuar==1:
      Accion=[1,8]
      print("Corte")
      break
    elif Actuar==2:
      Accion=[3,6]
      print("Puñalada")
      break
    elif Actuar==3:
      print("Boliar Machete")
      Accion=[4,4]
      break
    else:
      print("Opcion no valida")


def Tony():
  print("     _▲_")
  print("   ϟ  @  ϟ   ")
  print("  ϟ ϟ/|\ϟ ϟ    ")
  print("     / \    ")
def TonyHab():
  global Accion
  print("""Selecciona tu accion
    1. Chispazo
    2. Tormenta
    3. Sobrecarga""")
  while True:
    Actuar=int(input())
    if Actuar==1:
      Accion=[1,8]
      print("Chispazo")
      break
    elif Actuar==2:
      Accion=[2,6]
      print("Tormenta")
      break
    elif Actuar==3:
      print("Sobrecarga")
      Accion=[10,1]
      break
    else:
      print("Opcion no valida")

def Franco():
  print("     @     ")
  print("   //|!|\\     ")
  print("    / \    ")
  print("   /   |   ")
def FrancoHab():
  global Accion
  print("""Selecciona tu accion
    1. Puñetazo
    2. Desenfreno
    3. Cascar enemigos""")
  while True:
    Actuar=int(input())
    if Actuar==1:
      Accion=[1,9]
      print("Puñetazo")
      break
    elif Actuar==2:
      Accion=[5,4]
      print("Desenfreno")
      break
    elif Actuar==3:
      print("Cascar enemigos")
      Accion=[8,2]
      break
    else:
      print("Opcion no valida")
def Lobby():
  print("Bruno Balaso")
  print("     _Ω_")
  print("     @     ")
  print("    /|\/7 ==   ")
  print("    / \    ")
  print("----------------")
  print("Pepe peligro")
  print("     _-_")
  print("     @     ")
  print("    /|\~ı~=   ")
  print("    / \    ")
  print("----------------")
  print("Tony Tormenta")
  print("     _▲_")
  print("   ϟ  @  ϟ   ")
  print("  ϟ ϟ/|\ϟ ϟ    ")
  print("    / \    ")
  print("----------------")
  print("Franco Furia")
  print("     @     ")
  print("   //|!|\\     ")
  print("    / \    ")
  print("   /   |   ")

Lobby()


personaje_seleccionado1 = input(f"{J1}: ¿Quien soy?: Bruno, Pepe, Tony, Franco: ")
def personaje1():
  global personaje_seleccionado1, J1
  while True:
    if personaje_seleccionado1 == "Bruno":
      Bruno()
      break
    elif personaje_seleccionado1 == "Pepe":
      Pepe()
      break
    elif personaje_seleccionado1 == "Tony":
      Tony()
      break
    elif personaje_seleccionado1 == "Franco":
      Franco()
      break
    else:
      print("Personaje no válido")

def personaje1Hab():
  global personaje_seleccionado1
  while True:
    if personaje_seleccionado1 == "Bruno":
      BrunoHab()
      break
    elif personaje_seleccionado1 == "Pepe":
      PepeHab()
      break
    elif personaje_seleccionado1 == "Tony":
      TonyHab()
      break
    elif personaje_seleccionado1 == "Franco":
      FrancoHab()
      break
    else:
      print("Personaje no válido")

personaje1()

personaje_seleccionado2 = input(f"{J2}: ¿Quien soy?: Bruno, Pepe, Tony, Franco: ")
def personaje2():
  global personaje_seleccionado2, J2
  while True:
    if personaje_seleccionado2 == "Bruno":
      Bruno()
      break
    elif personaje_seleccionado2 == "Pepe":
      Pepe()
      break
    elif personaje_seleccionado2 == "Tony":
      Tony()
      break
    elif personaje_seleccionado2 == "Franco":
      Franco()
      break
    else:
      print("Personaje no válido")

def personaje2Hab():
  global personaje_seleccionado2
  while True:
    if personaje_seleccionado2 == "Bruno":
      BrunoHab()
      break
    elif personaje_seleccionado2 == "Pepe":
      PepeHab()
      break
    elif personaje_seleccionado2 == "Tony":
      TonyHab()
      break
    elif personaje_seleccionado2 == "Franco":
      FrancoHab()
      break
    else:
      print("Personaje no válido")

personaje2()
input("Presiona Enter para continuar")
clear_output()
def Estadisticas():
  R1 = [random.randint(10,2000),random.randint(10,2000)]
  R2 = [random.randint(10,2000),random.randint(10,2000)]
  return [R1, R2]
Stats1, Stats2 = Estadisticas()
urno():
bal J1, J2, Stats1, Stats2
andom.randint(1,2)
le True:
f Stats1[0] <= 0:
 print(J2, "GANAAAAA!")
 break
lif Stats2[0] <= 0:
 print(J1, "GANAAAAA!")
 break
rint("Vida de", J1, "es", Stats1[0], "Daño de ", J1 ,"es", Stats1[1
rint("Vida de", J2, "es", Stats2[0], "Daño de ", J2 ,"es", Stats2[1
f a % 2 == 0:
 print("Turno de", J1)
 personaje1()
 personaje1Hab()
 ValorDado = Dado()
 if ValorDado >= Accion[1]:
   Stats2[0] -= (Accion[0]*Stats1[1]) #Multiplico el multiplicador
   print("Fue efectivo!")
   a+=1
   input("Presiona Enter para continuar")
   clear_output()
 else:
   print("No fue efectivo")
   a+=1
   input("Presiona Enter para continuar")
   clear_output()
lse:
 print("Turno de", J2)
 personaje2()
 personaje2Hab()
 ValorDado = Dado()
 if ValorDado >= Accion[1]:
   Stats1[0] -= (Accion[0]*Stats2[1]) #Multiplico el multiplicador
   print("Fue efectivo!")
   a+=1
   input("Presiona Enter para continuar")
   clear_output()
 else:
   print("No fue efectivo")
   a+=1
   input("Presiona Enter para continuar")
   clear_output()

()
```

ranslate

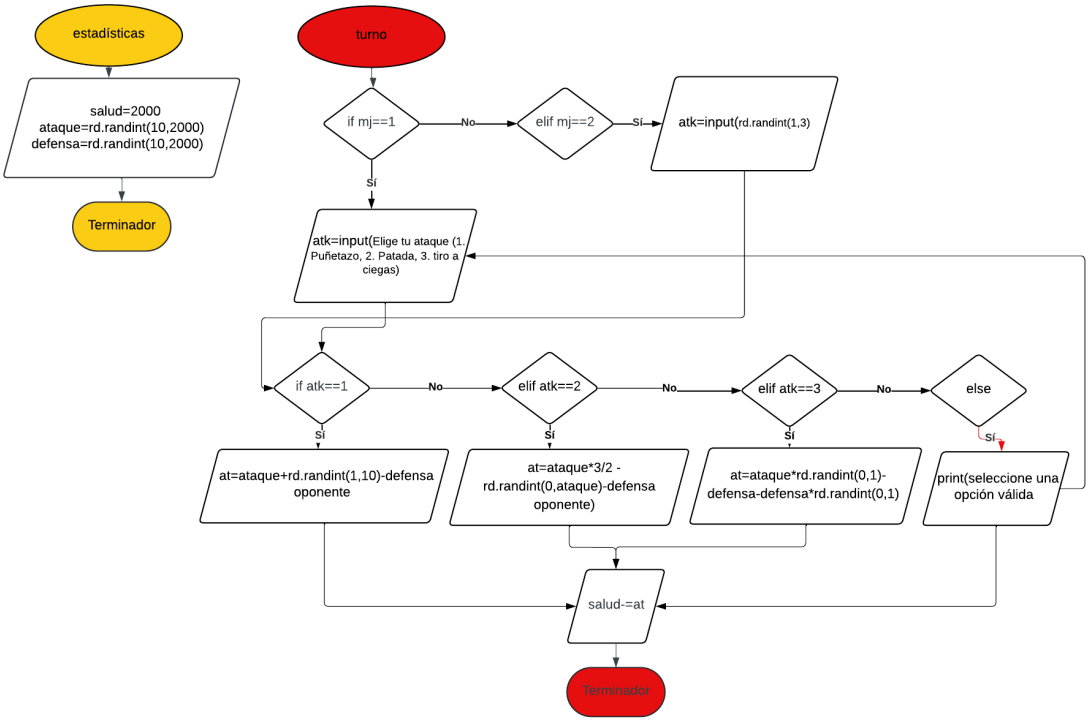
```
import random as rd

def calcular_daño(ataque, defensa):
  daño=ataque-defensa
  if daño<0:
    daño=0
  return daño

def turno_jugador(nombre_jugador, ataque_jugador, defensa_oponente):
  print(f"Turno de {nombre_jugador}!")
  while True:
    eleccion_ataque=input("Elige tu ataque (1. Puñetazo, 2. Patada giratoria, 3. tiro a
ciegas, 4. puñalada): ")
    if eleccion_ataque=='1':# el puñetazo es el ataque con menor rango de ataque, siendo este
el ataque + un número entre 1 y 10 - la defensa del oponente
      daño=calcular_daño(ataque_jugador + rd.randint(1,10), defensa_oponente)
      print(f"{nombre_jugador} da un puñetazo por {daño} de daño!")
      break
    elif eleccion_ataque=='2':#la patada giratoria tiene un rango de daño muy amplio, entre
1.5 del ataque - la defensa y 0.5 del ataque - la defensa
      daño=calcular_daño(ataque_jugador*3/2 - rd.randint(0,ataque_jugador), defensa_oponente)
      print(f"{nombre_jugador} da una patada giratoria por {daño} de daño!")
      break
    elif eleccion_ataque=='3': #el tiro a ciegas es un ataque que tiene un 50% de probabilidad
de acertar
      daño=calcular_daño(ataque_jugador*rd.randint(0,1),
defensa_oponente-(defensa_oponente*rd.randint(0,1)))#y si acierta tiene un 50% de probabilidad
de ignorar la defensa
      print(f"{nombre_jugador} da un tiro a ciegas por {daño} de daño!")
      break
    elif eleccion_ataque=='4': #la puñalada es un ataque que ignora la defensa del enemigo
      daño=calcular_daño(rd.randint(50,100), 0)#pero, no toma encuenta valor del ataque,
siempre el daño que causa esta entre 50, y 100
      print(f"{nombre_jugador} da una puñalada por {daño} de daño!")
      break
    else:
      print("Elección inválida. Por favor elige de nuevo.")
  return daño

def turno_computadora(ataque_computadora, defensa_oponente):
  print("Turno de la computadora")
  eleccion_ataque=rd.choice(['1','2','3','4'])#rd.choice escoge un elemento aleatorio de la
lista, lo que escoge el ataque de la computadora
  if eleccion_ataque=='1':
    daño=calcular_daño(ataque_computadora + rd.randint(1,10), defensa_oponente)
    print(f"La computadora da un puñetazo por {daño} de daño!")
  elif eleccion_ataque=='2':
    daño=calcular_daño(ataque_computadora*3/2 - rd.randint(0,ataque_computadora),
defensa_oponente)
    print(f"La computadora da una patada giratoria por {daño} de daño!")
  elif eleccion_ataque=='3':
    daño=calcular_daño(ataque_computadora*rd.randint(0,1),
defensa_oponente-(defensa_oponente*rd.randint(0,1)))
    print(f"La computadora da un tiro a ciegas por {daño} de daño!")
  elif eleccion_ataque=='4':
    daño=calcular_daño(rd.randint(50,100), 0)
    print(f"La computadora da una puñalada por {daño} de daño!")
```

```
  else:
    print("Elección inválida. Por favor elige de nuevo.")
  return daño

def jugar_juego():
  nombre_jugador=input("Introduce tu nombre: ")#se implementaron 2 modos de juego, JcJ y JcC+
  modo_juego=input("Elige el modo de juego (1. Jugador vs Jugador, 2. Jugador vs Computadora):
")
  salud_jugador=2000
  salud_oponente=2000#para que las partidas no acaben de un golpe, se estableció la salud en
2000
  ataque_jugador=rd.randint(10, 2000)
  ataque_oponente=rd.randint(10, 2000)
  a=int(round(ataque_oponente/2))
  b=int(round(ataque_oponente*9/10))#la defensa está en un rango entre la mitad y 9/10 del
ataque del otro jugador
  c=int(round(ataque_jugador*9/10))#esto se implementó para que sea más justo el
enfrentamiento
  d=int(round(ataque_jugador/2))
  defensa_jugador=rd.randint(a,b)
  defensa_oponente=rd.randint(d,c)
  if modo_juego=='1':
    nombre_oponente=input("Introduce el nombre del oponente: ")
  elif modo_juego=='2':
    nombre_oponente="La Computadora"
  else:
    print("Modo de juego inválido. Por favor elige 1 o 2.")
    print("se reiniciara el juego")
    jugar_juego()
  print(f"Bienvenido, {nombre_jugador}!")
  print(f"Salud de {nombre_jugador}: {salud_jugador}, defensa {defensa_jugador}, daño del
jugador: {ataque_jugador}")
  print(f"Salud de {nombre_oponente}: {salud_oponente}, defensa {defensa_oponente}, daño del
oponente: {ataque_oponente}")
  while salud_jugador>0 and salud_oponente>0:
    if modo_juego=='1':
      salud_oponente-=turno_jugador(nombre_jugador, ataque_jugador, defensa_oponente)
      if salud_oponente<=0:
        break
      salud_jugador-=turno_jugador(nombre_oponente, ataque_oponente, defensa_jugador)
    elif modo_juego=='2':
      salud_oponente-=turno_jugador(nombre_jugador, ataque_jugador, defensa_oponente)
      if salud_oponente<=0:
        break
      salud_jugador-=turno_computadora(ataque_oponente, defensa_jugador)
    else:
      print("Modo de juego inválido. Por favor elige 1 o 2.")
      break

  if salud_jugador<=0:
    print("Gana",nombre_oponente)
  elif salud_oponente<=0:
    print("Gana",nombre_jugador)


jugar_juego()
```

Diagrama de Flujo



funciones del diagrama de flujo



Implementación

La implementación de lo visto en clase fue principalmente buscar formas para facilitar un trabajo extenso en algo mucho más conciso, siendo algunas las variables utilizadas para llamar funciones específicas o usar estas mismas funciones para organizar de manera concisa toda la parte ejecutable del programa. Otro tema que también es vital para el funcionamiento del juego son las listas pues estas contienen toda la información numérica de los personajes y es tanto usada como cambiada dependiendo de lo que ocurre. Finalmente, el uso de la biblioteca random para añadir emoción con el aspecto de azar al juego.

Como conclusión, el desarrollo de este miniproyecto nos brindó una oportunidad para profundizar en las funciones básicas de Python y mejorar nuestro pensamiento lógico. Aplicamos conceptos fundamentales como estructuras de control, variables y funciones, lo que nos permitió crear un programa funcional e interactivo. Además, nos desafiamos a buscar soluciones creativas y a trabajar en equipo, fortaleciendo nuestras habilidades de colaboración.